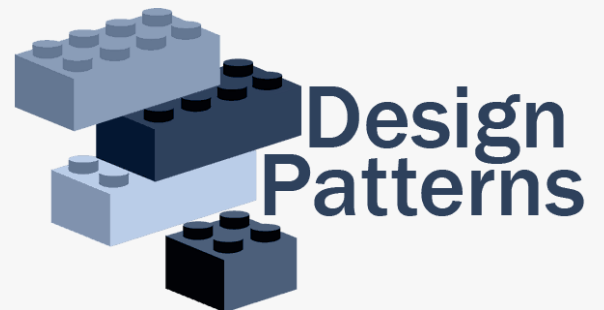
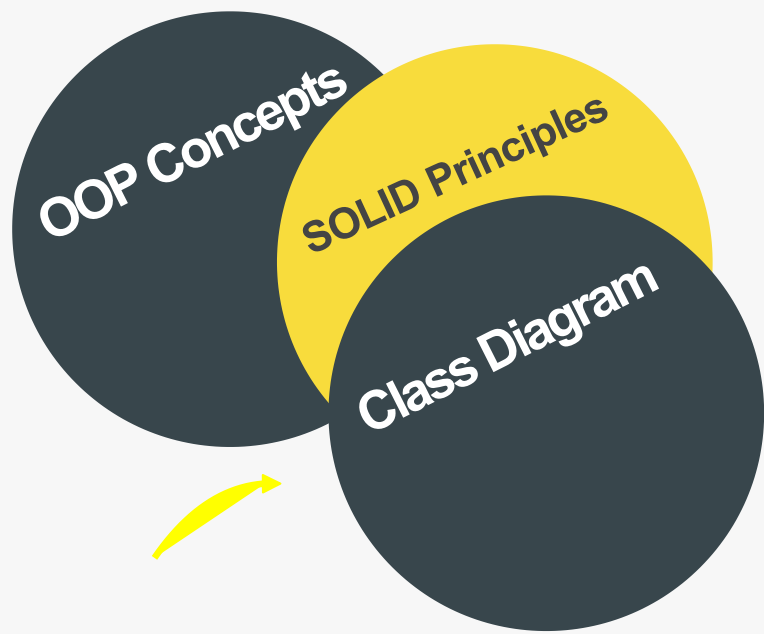


Slides for...



Design Patterns

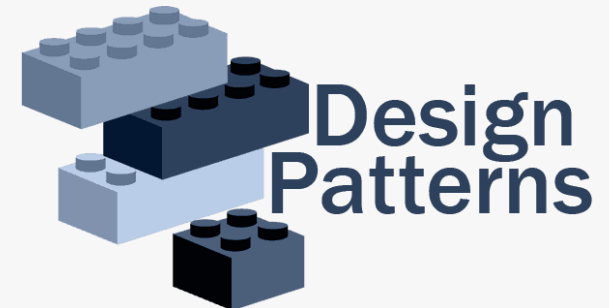
Eng / Ryhab Farouq 🧐



WHAT IS Object Oriented Programming?

OOP

Is a programming paradigm that relies on the concept of classes and objects, It is used to structure a software program into simple, reusable pieces of code blueprints (usually called classes), which are used to create individual instances of objects.



Inheritance (is-a)

Refers to the capability of creating a new class from an existing class. It is a relationship between classes where one class is a parent (super or base) of another (Child, subclass, derived).

👉 Inheritance Hierarchy

The relationship between super classes and subclasses.

👉 Multi-Level Inheritance

In Multilevel Inheritance a derived class can also be inherited by another class.

Association (has-a)

Association establish relationship between two separate classes through their objects.

- 👉 **Weak Association (Aggregation)**
- 👉 **Strong Association (Composition)**

x **Dynamic (Late) Binding**

Dynamic binding also called dynamic dispatch is the process of linking method call to a specific sequence of code at run-time, It means that the code to be executed for a specific method call is not known until run-time.

x **Virtual Function**

A virtual function is a member function that you expect to be redefined in derived classes.

x **Abstract Method (Pure Virtual Function)**

An abstract method is a method that is declared, but contains no implementation, without body, It is needed to be overridden by the subclasses .

x Abstract Class

Abstract classes are distinguished by the fact that you may not directly construct objects from them, An abstract class may have one or more abstract methods, Abstract classes may not be instantiated.

```
//Created an abstract class (constructor function)
function Employee()
{
  this.empName= "empName";
  if(this.constructor === Employee){
    throw new Error("FYI: Instance of Abstract class cannot be instantiated");
  }
} ;
// Method (function) of our abstract class
Employee.prototype.display=function()
{ return this.empName; }
var employee = new Employee();
```

x Concrete Class

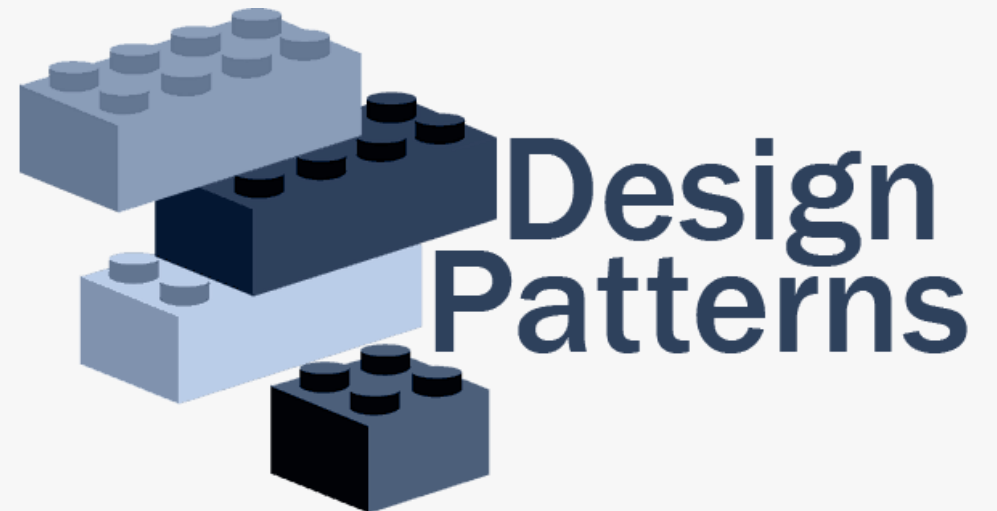
Any class can be initiating objects directly from it, it is not abstract class.

x **Interface**

capable of describing the wide range of shapes that JavaScript objects can take.

x **Enum**

special data types that set variables as a set of predefined constants.





SOLID Principles

The **SOLID** design principles help us create maintainable, reusable, and flexible software designs.

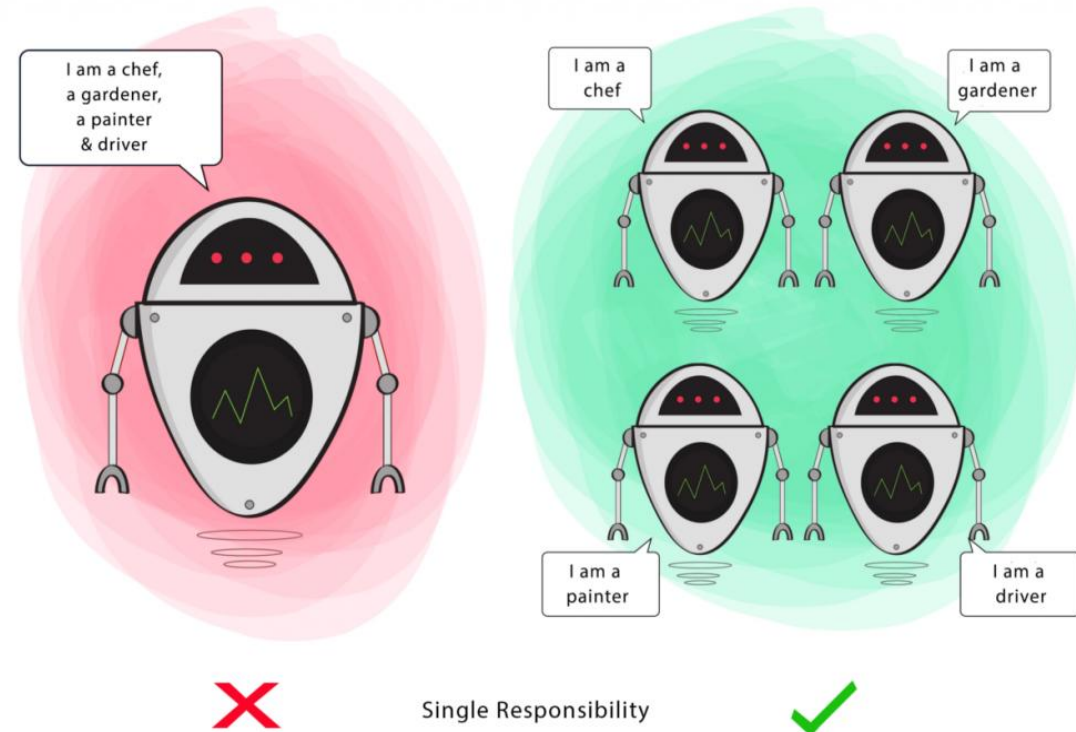
Each letter in the acronym **SOLID** stands for a specific principle.

- 👉 **S**ingle responsibility principle
- 👉 **O**pen–closed principle
- 👉 **L**iskov substitution principle
- 👉 **I**nterface segregation principle
- 👉 **D**ependency inversion principle

Single Responsibility Principle

“Do one thing and do it well”

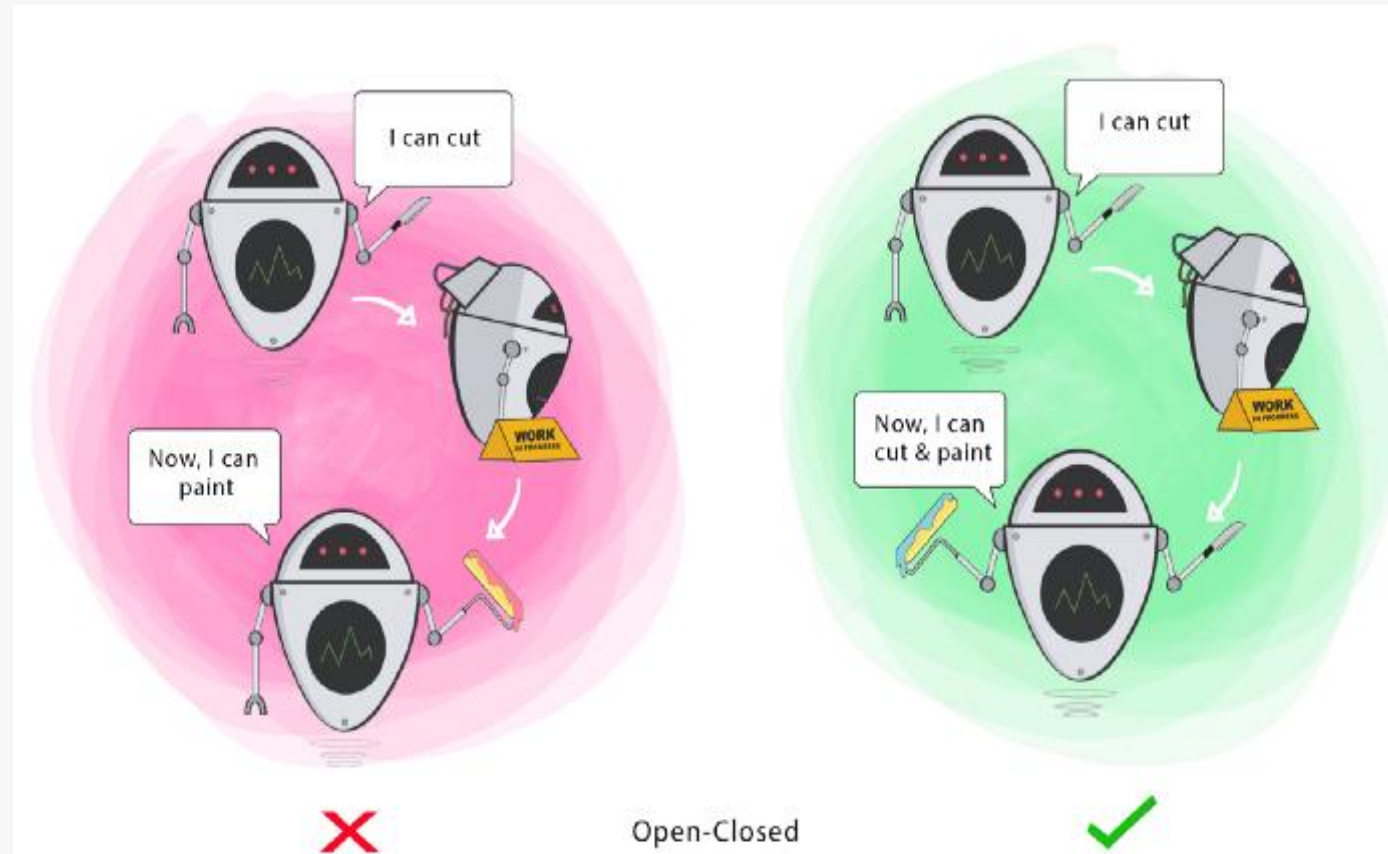
A class should have only one reason to change ,
meaning that a class should have only one job .



Open/Closed Principle

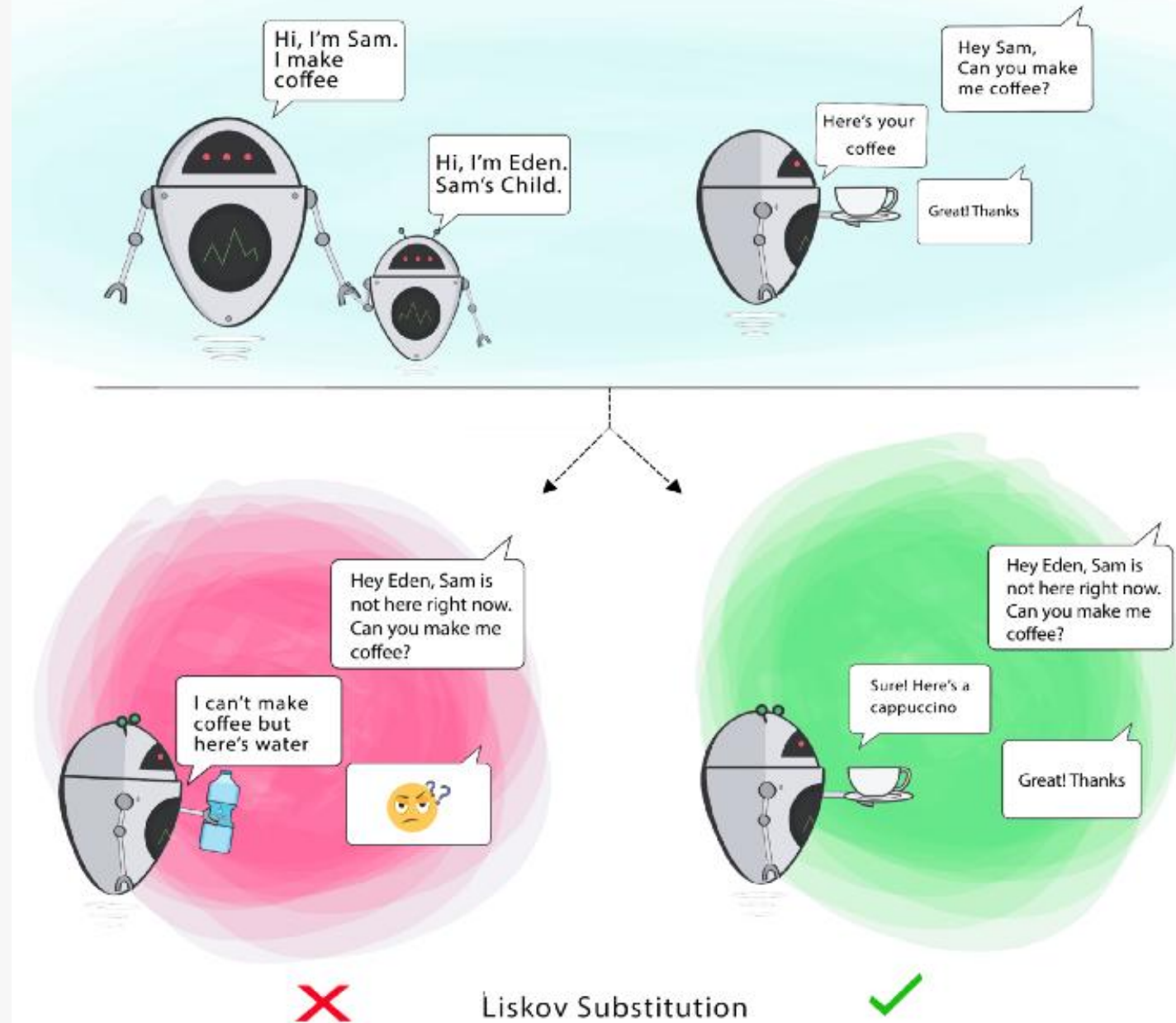
“Software entities classes , modules , functions should be opened for extension , but closed for modification”

Any new functionality should be done by adding new classes instead of changing existing one .



Liskov Substitution Principle

“If you have class B inherits from class A then class A should be replaced by class B without any changes ”



Interface Segregation Principle

“Clients should not be forced to depend on methods they don’t use”

👉 Avoid fat interface



X

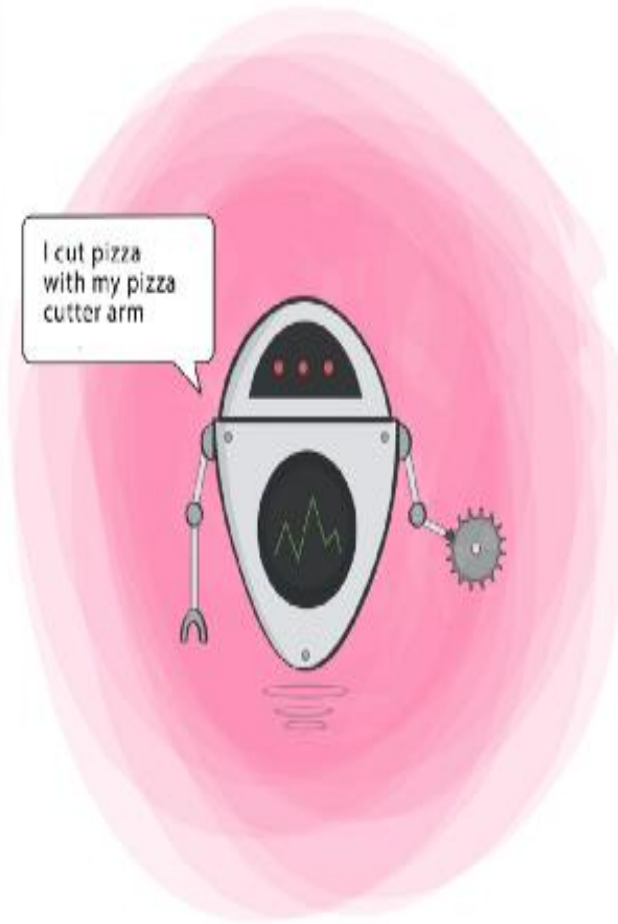


✓

Interface Segregation

Dependency Inversion Principle

“High level module should not depend upon low level modules , both should depend upon abstractions ”



✗



✓

Dependency Inversion

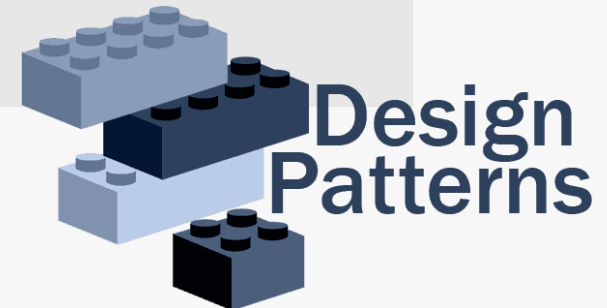
Design Patterns

DP

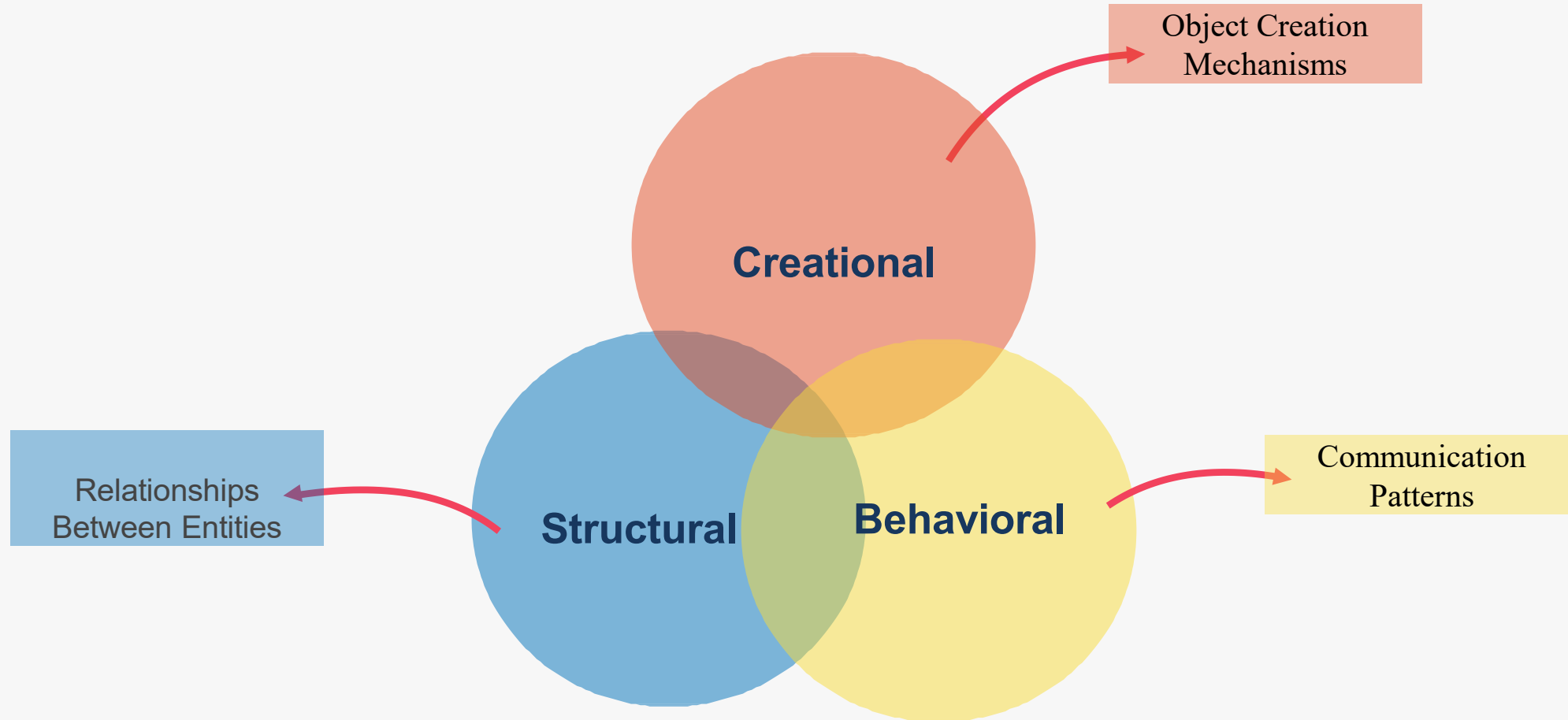
X Design Pattern is not code. It is in fact an approach or a model that can be used to solve a problem.

X Design Patterns are about design and interaction of objects and they provide reusable solutions for solving common design problems.

X Design pattern consists of various co-operating objects (classes, relationships, etc).



Design Pattern Categories



Design Pattern Categories

Creational Design Patterns

Singleton

Prototype

Builder

Factory Method

Abstract Factory

Structural Design Patterns

Adapter

Bridge

Composite

Decorator

Facade

Flyweight

Proxy

Behavioral Design Patterns

Strategy

State

Template Method

Visitor

Observer

Memento

Mediator

Iterator

Interpreter

Command

Chain of Resp.

Creational Design Patterns

Singleton

Prototype

Builder

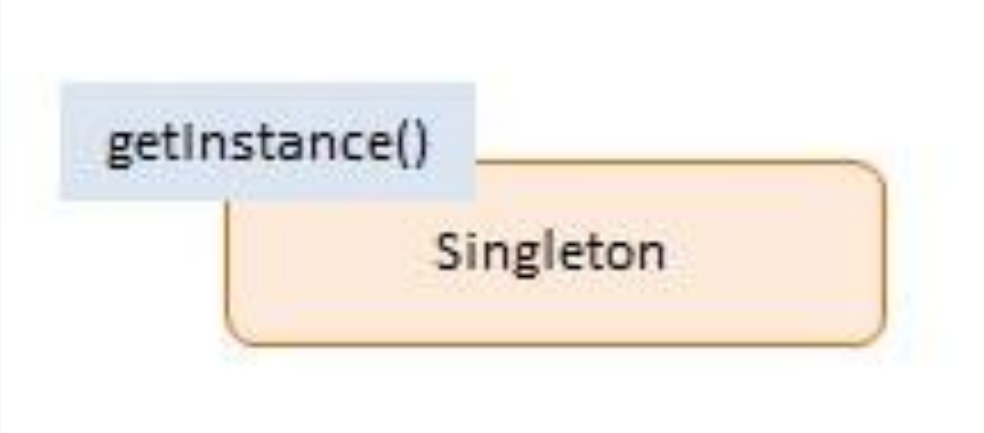
Factory Method

Abstract Factory

Singletons are useful in situations where system-wide actions need to be coordinated from a single central place.

`getInstance()`

Singleton

A diagram illustrating the Singleton pattern. It features a light blue rectangular box labeled 'getInstance()' with a pointer line extending from its bottom edge to an orange rounded rectangular box labeled 'Singleton'. The 'Singleton' box has a thin black border and a subtle drop shadow.

Creational Design Patterns

Singleton

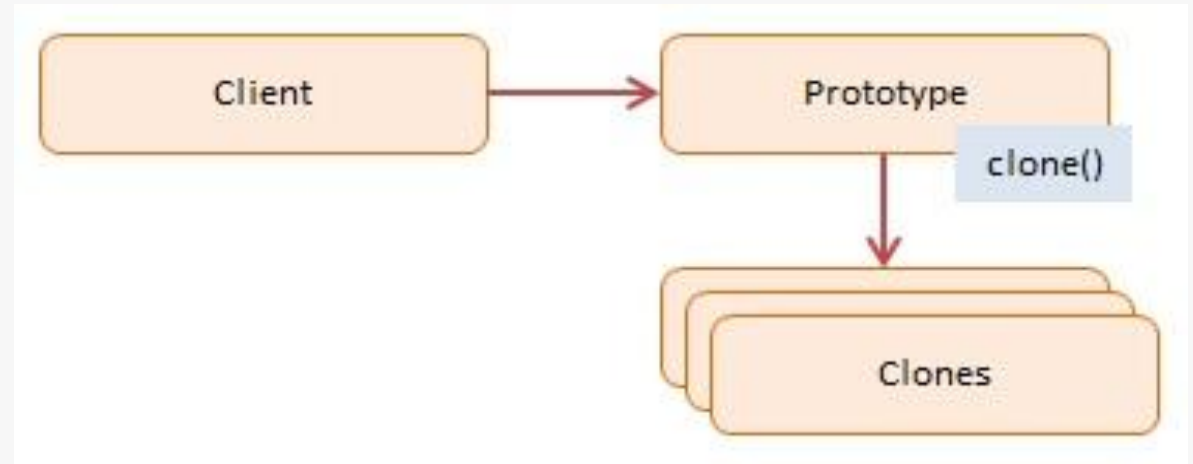
Prototype

Builder

Factory Method

Abstract Factory

The Prototype Pattern creates new objects, but rather than creating non-initialized objects it returns objects that are initialized with values it copied from a prototype ,The Prototype pattern is also referred to as the Properties pattern.



Creational Design Patterns

Singleton

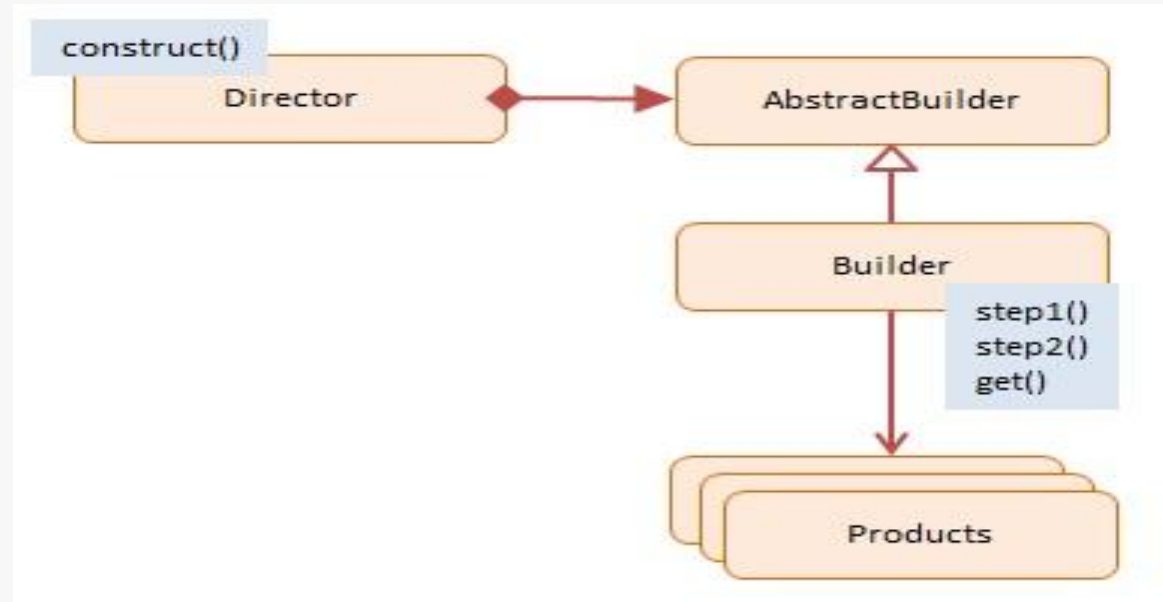
Prototype

Builder

Factory Method

Abstract Factory

The Builder pattern allows a client to construct a complex object by specifying the type and content only, Construction details are hidden from the client entirely.



Creational Design Patterns

Singleton

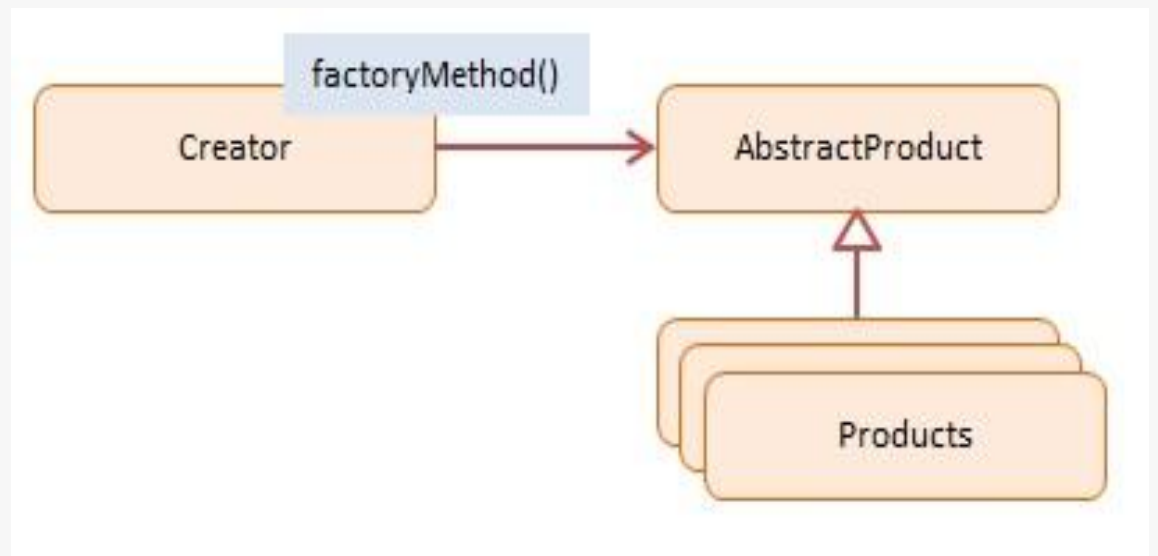
Prototype

Builder

Factory Method

Abstract Factory

A Factory Method creates new objects as instructed by the client. One way to create objects in JavaScript is by invoking a constructor function with the new operator. There are situations however, where the client does not, or should not, know which one of several candidate objects to instantiate. The Factory Method allows the client to delegate object creation while still retaining control over which type to instantiate.



Creational Design Patterns

Singleton

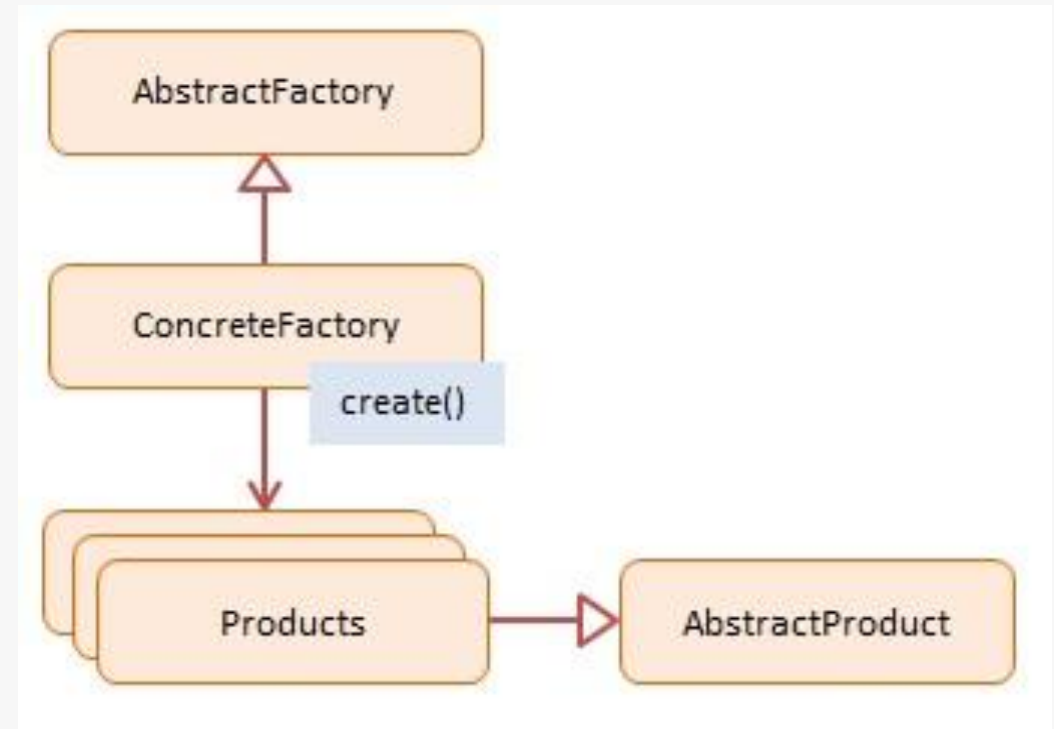
Prototype

Builder

Factory Method

Abstract Factory

An *Abstract Factory* creates objects that are related by a common theme. In object-oriented programming a Factory is an object that creates other objects. An Abstract Factory has abstracted out a theme which is shared by the newly created objects.





Day 2

Structural Design Patterns

Proxy

Decorator

Facade

Adapter

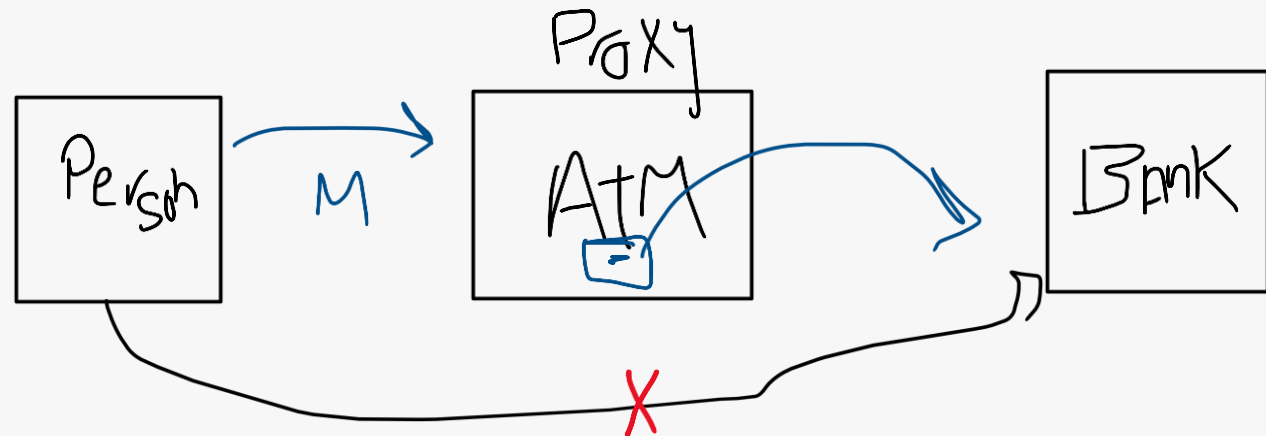
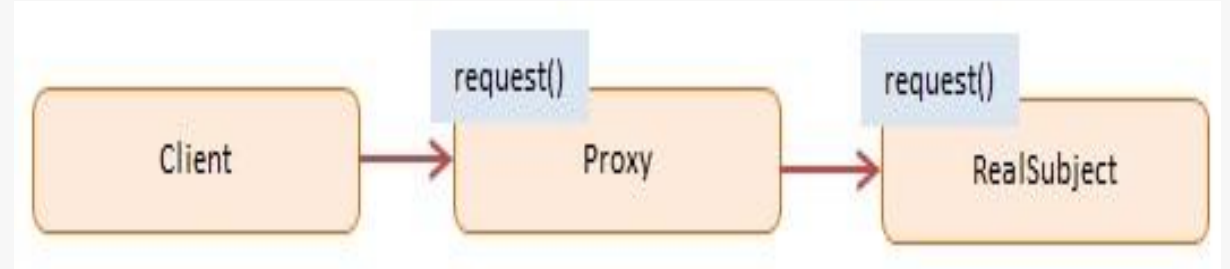
Flyweight

Composite

Bridge

The Proxy pattern provides a surrogate or placeholder object for another object and controls access to this other object.

The proxy is the object that is being called by the client to access the real object behind the scene.



Structural Design Patterns

Proxy

Decorator

Facade

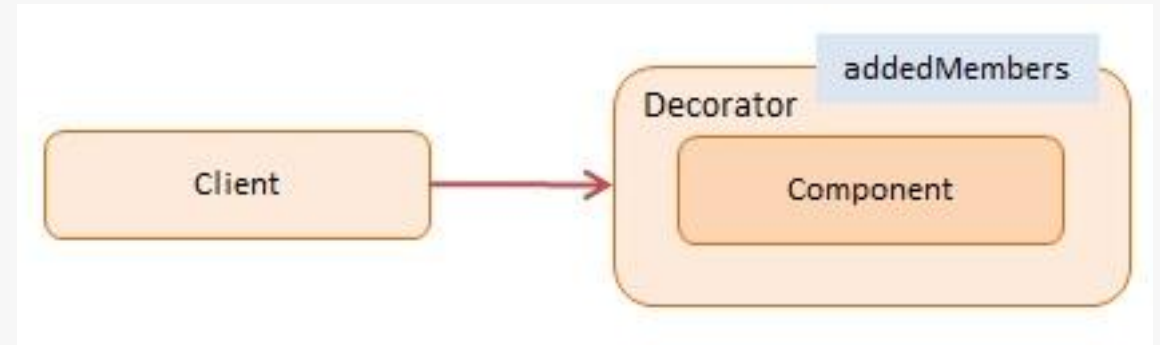
Adapter

Flyweight

Composite

Bridge

Decorators provide flexibility to statically typed languages by allowing runtime changes as opposed to inheritance which takes place at compile time. JavaScript, however, is a dynamic language and the ability to extend an object at runtime is baked into the language itself.



Structural Design Patterns

Proxy

Decorator

Facade

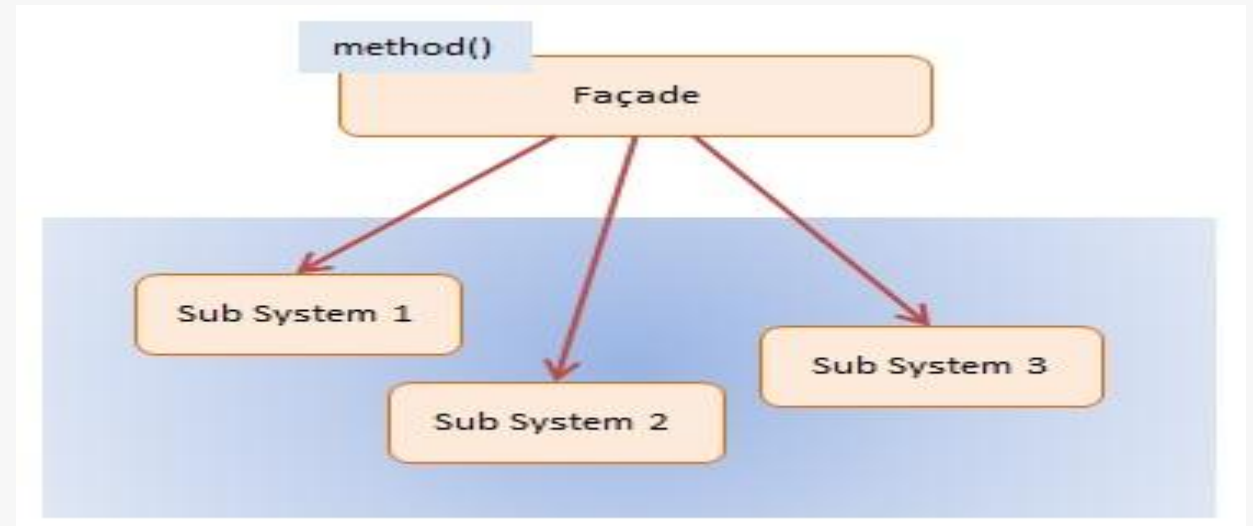
Adapter

Flyweight

Composite

Bridge

The Façade pattern provides an interface which shields clients from complex functionality in one or more subsystems. It is a simple pattern that may seem trivial but it is powerful and extremely useful. It is often present in systems that are built around a multi-layer architecture.



Structural Design Patterns

Proxy

Decorator

Facade

Adapter

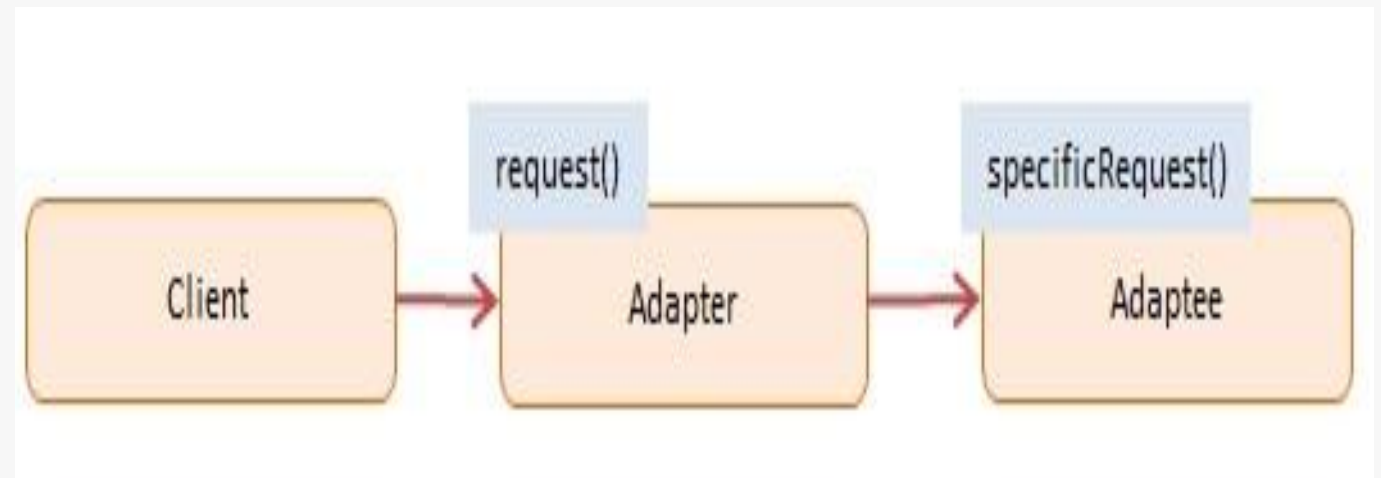
Flyweight

Composite

Bridge

The Adapter pattern translates one interface (an object's properties and methods) to another. Adapters allows programming components to work together that otherwise wouldn't because of mismatched interfaces. The Adapter pattern is also referred to as the Wrapper Pattern.

Adapter pattern involves a single class as adaptor which is responsible for communication between two independent or incompatible interfaces.



Structural Design Patterns

Proxy

Decorator

Facade

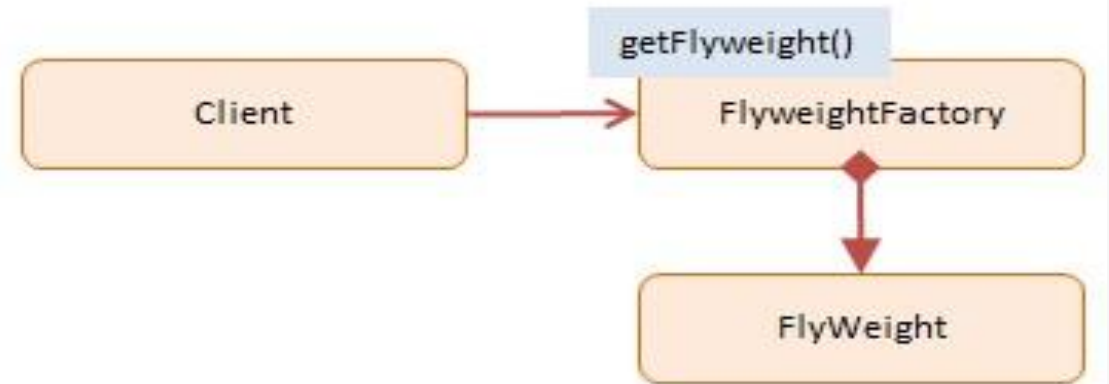
Adapter

Flyweight

Composite

Bridge

The Flyweight pattern conserves memory by sharing large numbers of fine-grained objects efficiently. Shared flyweight objects are immutable, that is, they cannot be changed as they represent the characteristics that are shared with other objects.



Structural Design Patterns

Proxy

Decorator

Facade

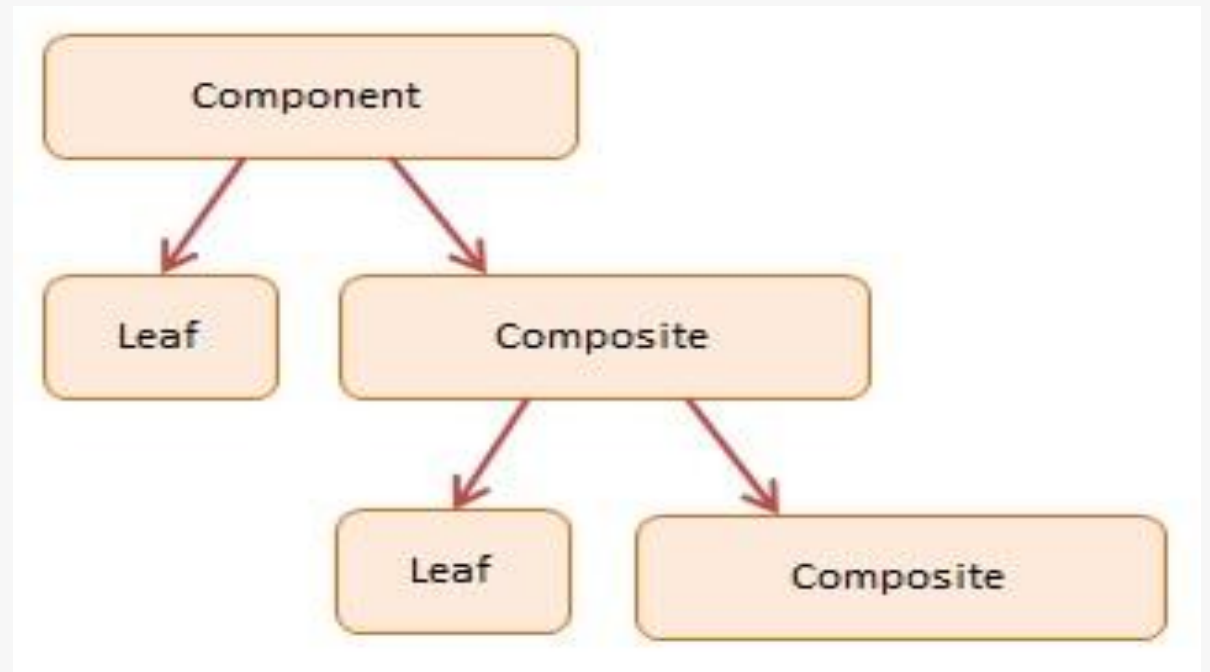
Adapter

Flyweight

Composite

Bridge

The Composite pattern allows the creation of objects with properties that are primitive items or a collection of objects. Each item in the collection can hold other collections themselves, creating deeply nested structures.



Structural Design Patterns

Proxy

Decorator

Facade

Adapter

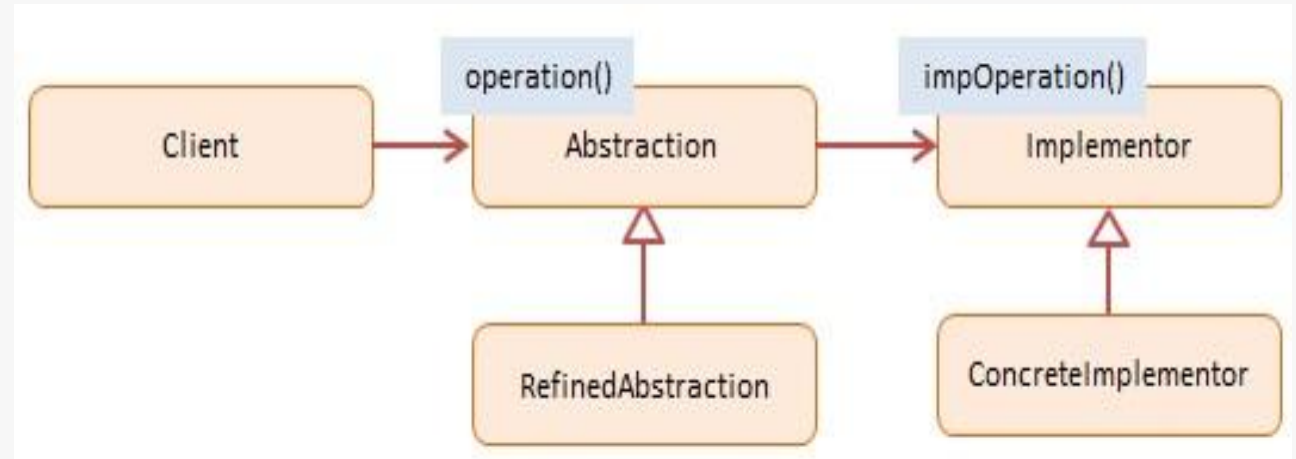
Flyweight

Composite

Bridge

The *Bridge* pattern allows two components, a client and a service, to work together with each component having its own interface. Bridge is a high-level architectural pattern and its main goal is to write better code through two levels of abstraction. It facilitates very loose coupling of objects. It is sometimes referred to as a double Adapter pattern.

self-study + demo



Behavioral Design Patterns

Strategy

State

Template Method

Visitor

Observer

Memento

Mediator

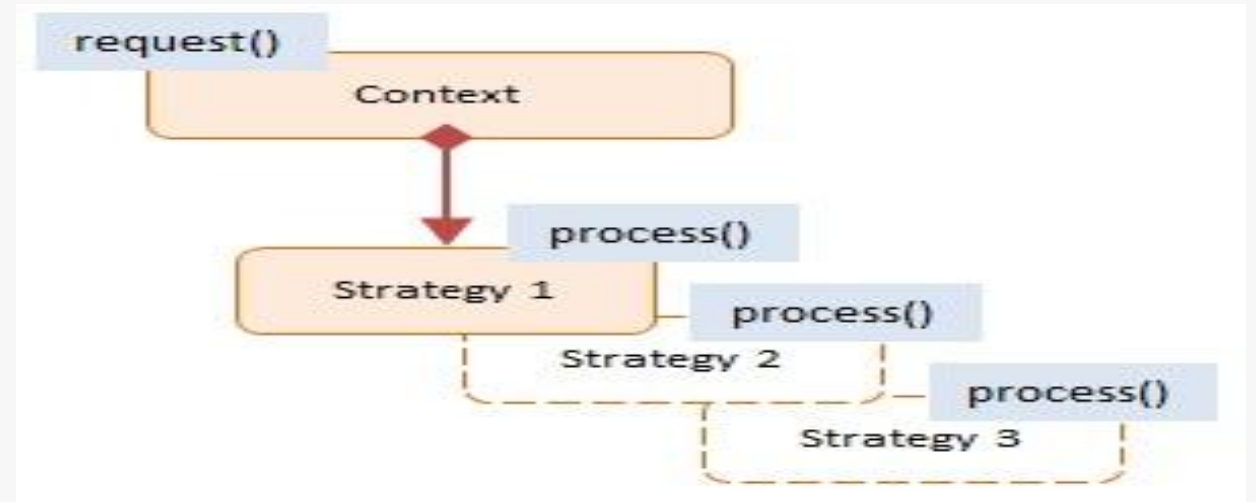
Iterator

Interpreter

Command

Chain of Resp.

The Strategy pattern encapsulates alternative algorithms (or strategies) for a particular task. It allows a method to be swapped out at runtime by any other method (strategy) without the client realizing it. Essentially Strategy is a group of algorithms that are interchangeable.



Behavioral Design Patterns

Strategy

State

Template Method

Visitor

Observer

Memento

Mediator

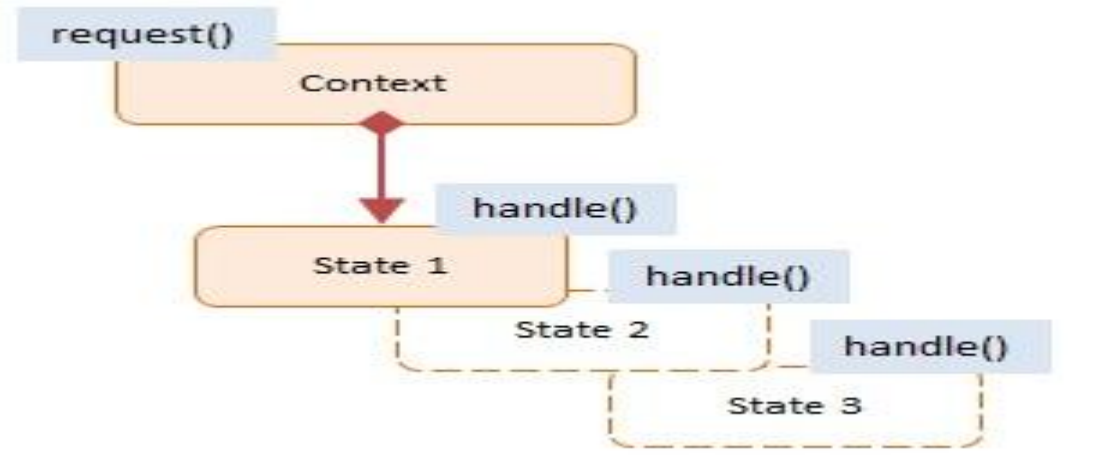
Iterator

Interpreter

Command

Chain of Resp.

The State pattern provides state-specific logic to a limited set of objects in which each object represents a particular state, This is best explained with an example.



Tasks

1- create decorator pattern to make teacher add dynamic property salary ,Nationality , street

2- create proxy design pattern to get countries if cached or not

3- create bridge design pattern where TV and speaker share the same interface of [increase_volume , decrease_volume and mute] , two bridge functions supports increase and decrease but one of them support mute also

4- create composite design pattern where box of books which have a number of pages and show the hierarchy